DEPARTAMENTO
DE INDUSTRIAS
UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Informe de Laboratorio 5:

Entregable SQL (2/2): base de datos de Andes Bike

Diego Victoriano Rodríguez
Ingeniería Civil Industrial, 21.540.820-5, *diego.victoriano@usm.cl*

ICN292 - Sistemas de Información para la Gestión.
Paralelo: 101

Profesores: José Miguel González P. | José Luis Sáez T. | Patricio Cifuentes

Ayudantes: Valeria Bobadilla |Catalina Bodaleo | Luis Contreras |Benjamín Jara

Repositorio GitHub:
https://github.com/diegovictorianor/ICN292-Lab5-Victoriano-Diego

noviembre, 2026

Campus Vitacura

Consulta 1: vendedores con su cantidad de ventas

```sql
SELECT v.nombre,
       COUNT(ve.id_venta) AS cantidad_ventas
FROM vendedores v
LEFT JOIN ventas ve ON v.id_vendedor = ve.id_vendedor
GROUP BY v.id_vendedor, v.nombre
ORDER BY cantidad_ventas DESC, v.nombre;
```

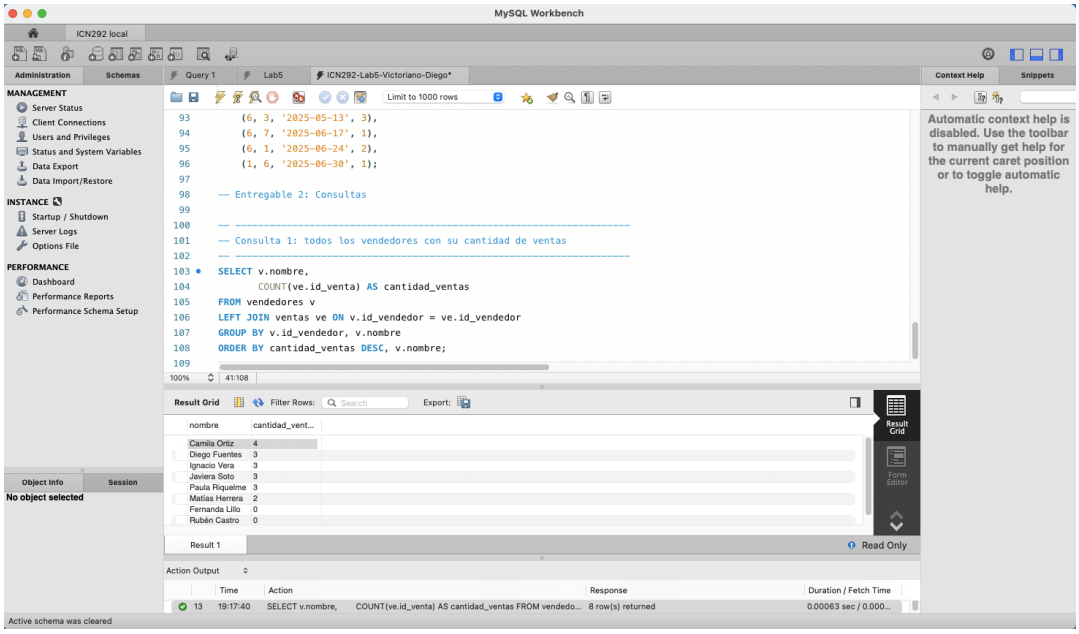


Figura 1: Resultado de la consulta 1 en MySQL Workbench.

Consulta 2: ventas y unidades vendidas por tienda

```sql
SELECT t.nombre,
       COUNT(ve.id_venta)              AS total_ventas,
       IFNULL(SUM(ve.unidades), 0)     AS total_unidades
FROM tiendas t
LEFT JOIN vendedores v  ON t.id_tienda   = v.id_tienda
LEFT JOIN ventas     ve ON v.id_vendedor = ve.id_vendedor
GROUP BY t.id_tienda, t.nombre
ORDER BY total_unidades DESC, t.nombre;
```

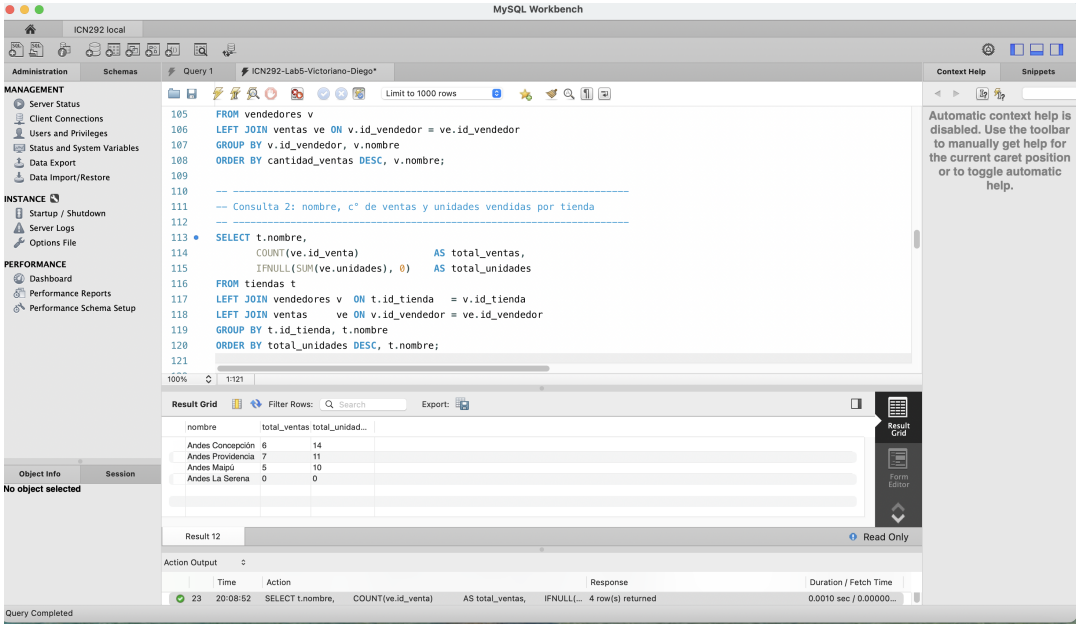


Figura 2: Resultado de la consulta 2 en MySQL Workbench.

Consulta 3: tiendas sin ningún vendedor asignado

```sql
SELECT t.nombre, t.comuna
FROM tiendas t
LEFT JOIN vendedores v ON t.id_tienda = v.id_tienda
WHERE v.id_vendedor IS NULL;
```

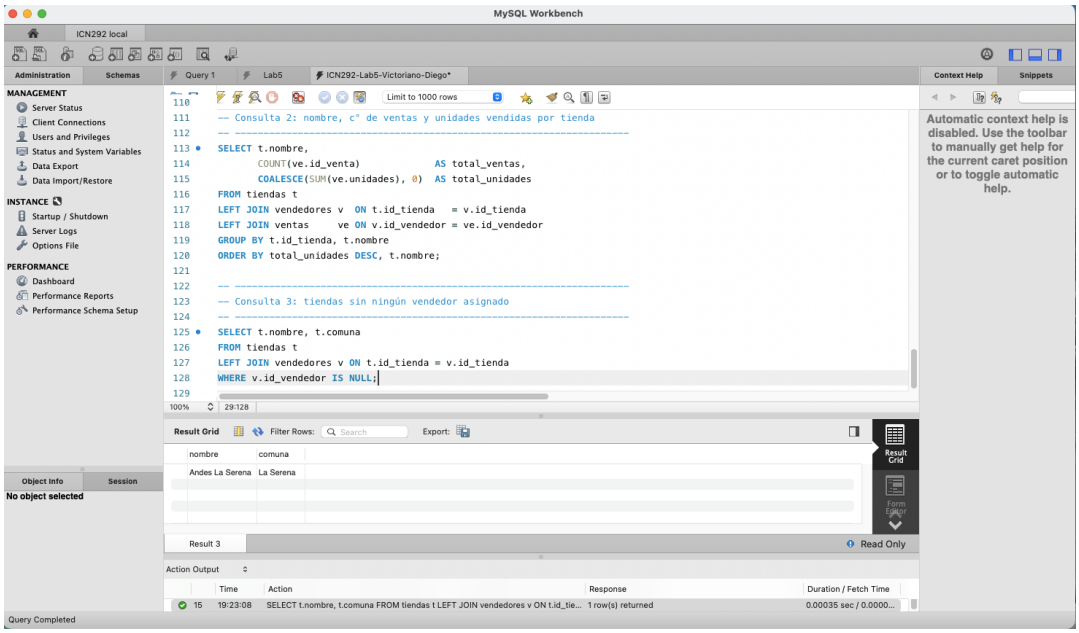


Figura 3: Resultado de la consulta 3 en MySQL Workbench.

Consulta 4: modelos sin ventas en junio de 2025

```sql
SELECT b.modelo, b.categoria
FROM bicicletas b
LEFT JOIN ventas ve ON b.id_bicicleta = ve.id_bicicleta
                   AND ve.fecha >= '2025-06-01'
                   AND ve.fecha <  '2025-07-01'
WHERE ve.id_venta IS NULL
ORDER BY b.modelo;
```

La condición del mes viene en el `ON` del `LEFT JOIN`. Si la hubiera puesto en el `WHERE`, ese filtro descartaría las filas sin pareja (con `fecha` NULL) y la consulta devolvería cero filas.

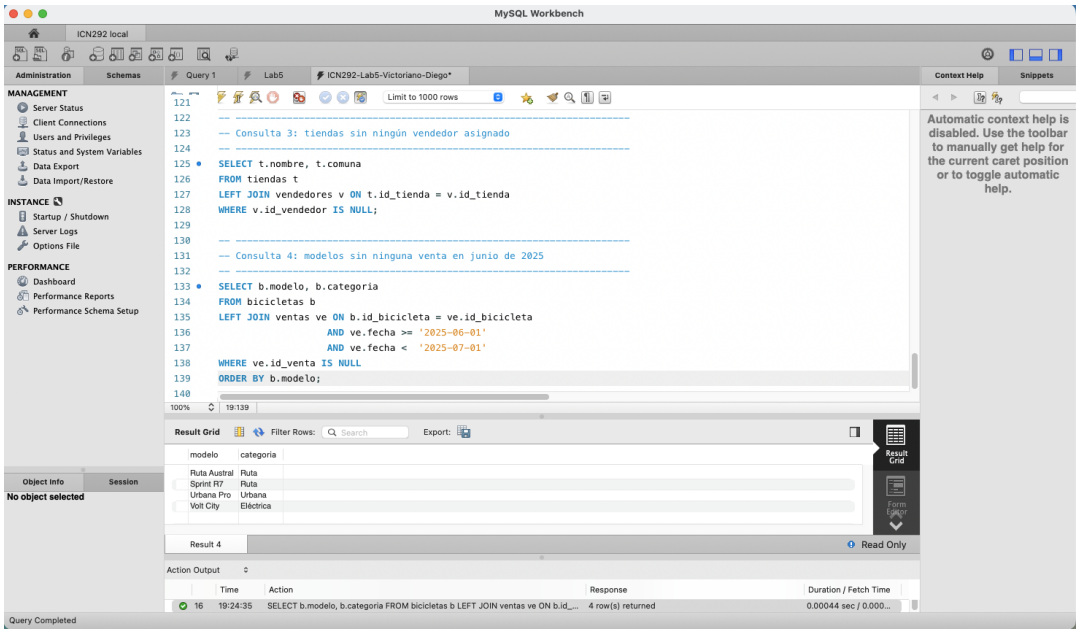


Figura 4: Resultado de la consulta 4 en MySQL Workbench.

Consulta 5: vendedores sin ventas y su tienda

```sql
SELECT v.nombre AS vendedor,
       t.nombre AS tienda
FROM vendedores v
LEFT JOIN tiendas t  ON v.id_tienda   = t.id_tienda
LEFT JOIN ventas  ve ON v.id_vendedor = ve.id_vendedor
WHERE ve.id_venta IS NULL
ORDER BY v.nombre;
```

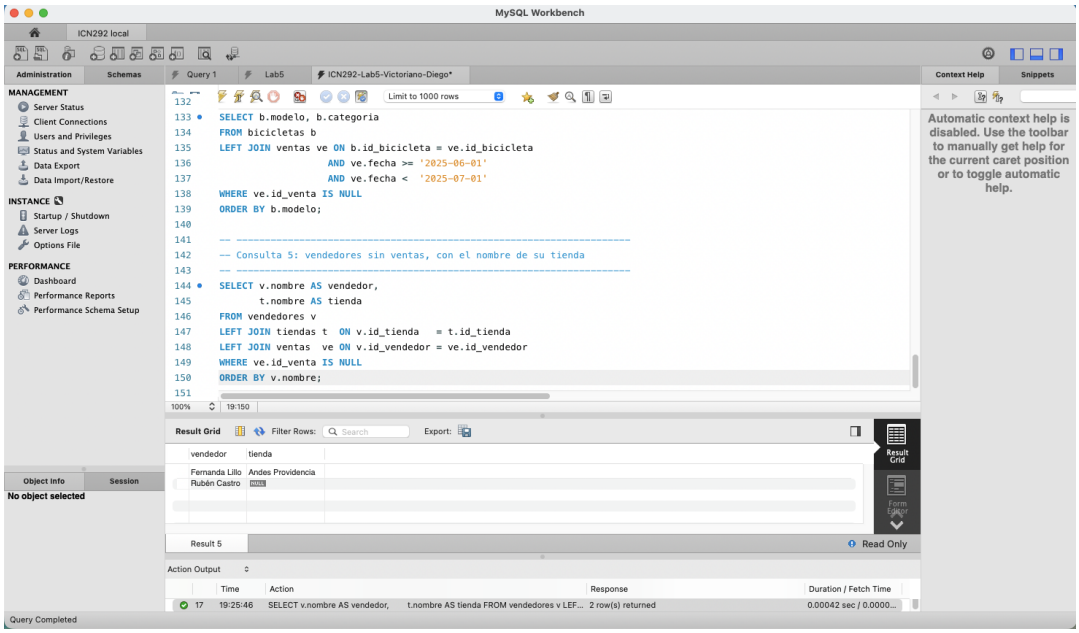


Figura 5: Resultado de la consulta 5 en MySQL Workbench.

Consulta 6: pares de vendedores de la misma tienda

```sql
SELECT v1.nombre         AS vendedor_1,
       v1.fecha_ingreso AS ingreso_1,
       v2.nombre         AS vendedor_2,
       v2.fecha_ingreso AS ingreso_2,
       t.nombre          AS tienda
FROM vendedores v1
JOIN vendedores v2 ON v1.id_tienda = v2.id_tienda
                  AND (v1.fecha_ingreso < v2.fecha_ingreso
                       OR (v1.fecha_ingreso = v2.fecha_ingreso
                           AND v1.id_vendedor < v2.id_vendedor))
JOIN tiendas t ON v1.id_tienda = t.id_tienda
ORDER BY t.nombre, v1.fecha_ingreso, v2.fecha_ingreso;
```

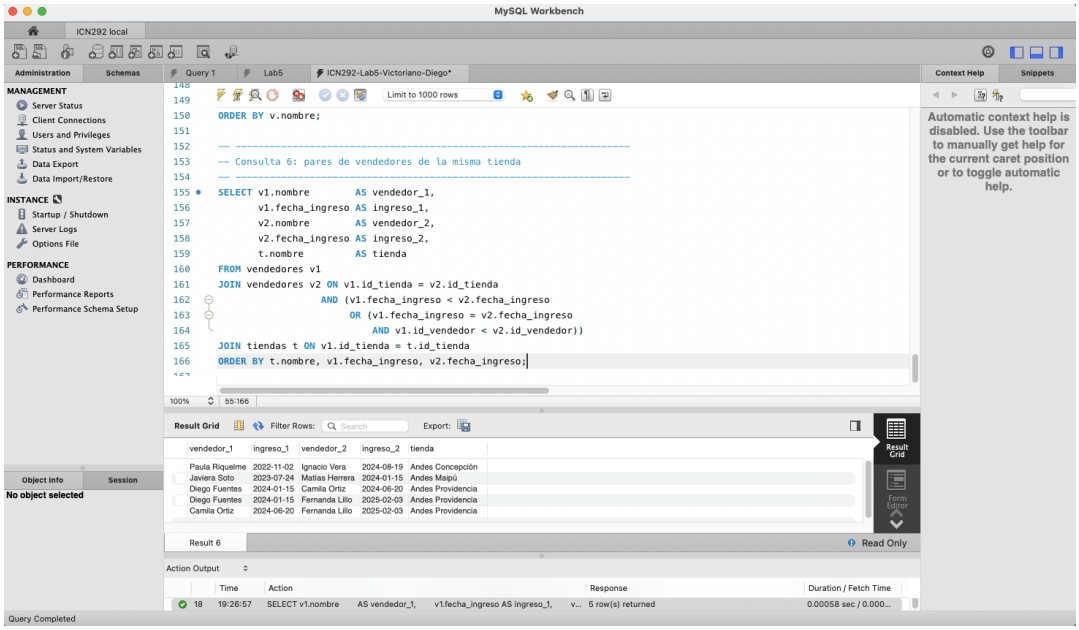


Figura 6: Resultado de la consulta 6 en MySQL Workbench.

Consulta 7: monto vendido en Montaña y Ruta por tienda

```sql
SELECT t.nombre,
       SUM(ve.unidades * b.precio) AS monto_total
FROM tiendas t
JOIN vendedores v  ON t.id_tienda    = v.id_tienda
JOIN ventas     ve ON v.id_vendedor  = ve.id_vendedor
JOIN bicicletas b  ON ve.id_bicicleta = b.id_bicicleta
WHERE b.categoria IN ('Montaña', 'Ruta')
GROUP BY t.id_tienda, t.nombre
HAVING SUM(ve.unidades * b.precio) > 2000000
ORDER BY monto_total DESC;
```

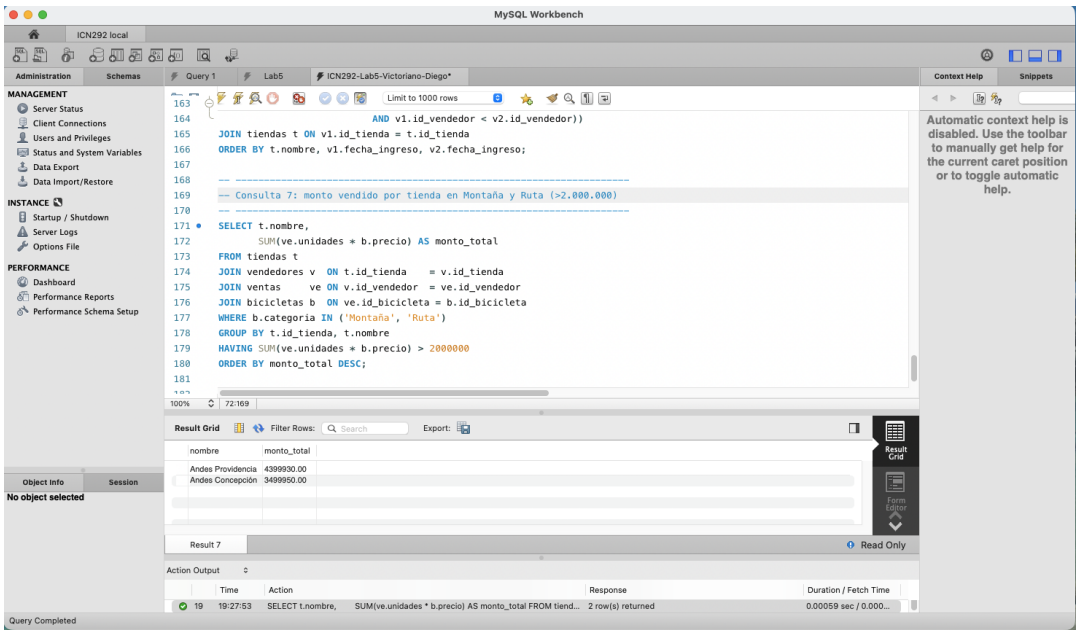


Figura 7: Resultado de la consulta 7 en MySQL Workbench.

Consulta 8: bicicletas sobre el precio promedio del catálogo

```sql
SELECT modelo, precio
FROM bicicletas
WHERE precio > (SELECT AVG(precio) FROM bicicletas)
ORDER BY precio DESC;
```

El promedio del catálogo es $657.490 (verificado en la Figura 9). en esta consulta se obtuvo mediante la subconsulta `SELECT AVG(precio) FROM bicicletas`, que promedia el precio de los 8 modelos de la tabla `bicicletas`.

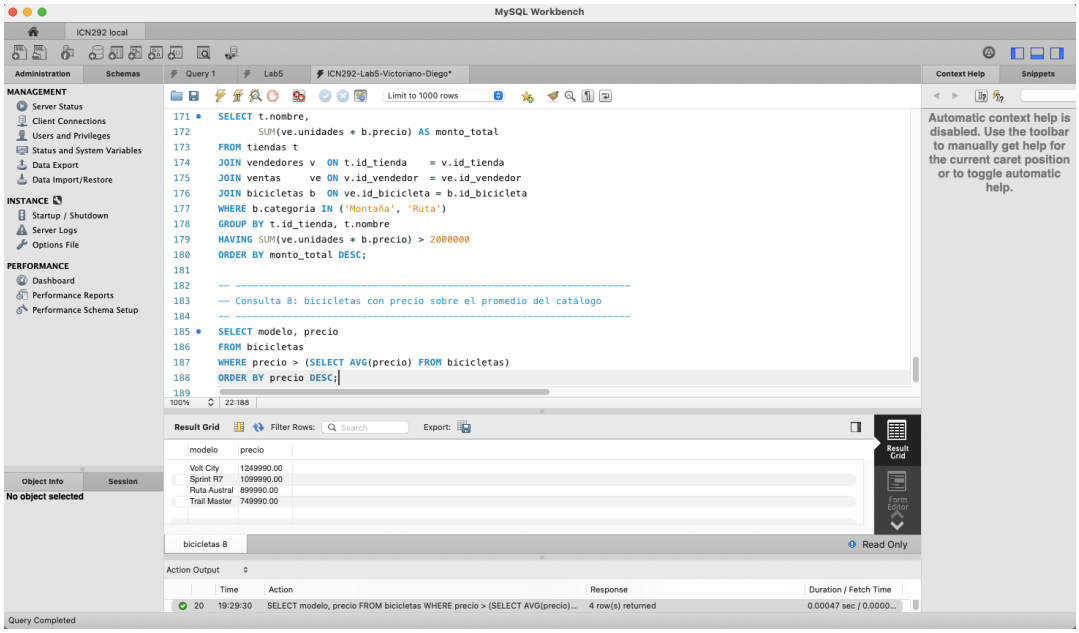


Figura 8: Resultado de la consulta 8 en MySQL Workbench.

Verificación del promedio del catálogo

```
SELECT AVG(precio) AS promedio_catalogo
FROM bicicletas;
```

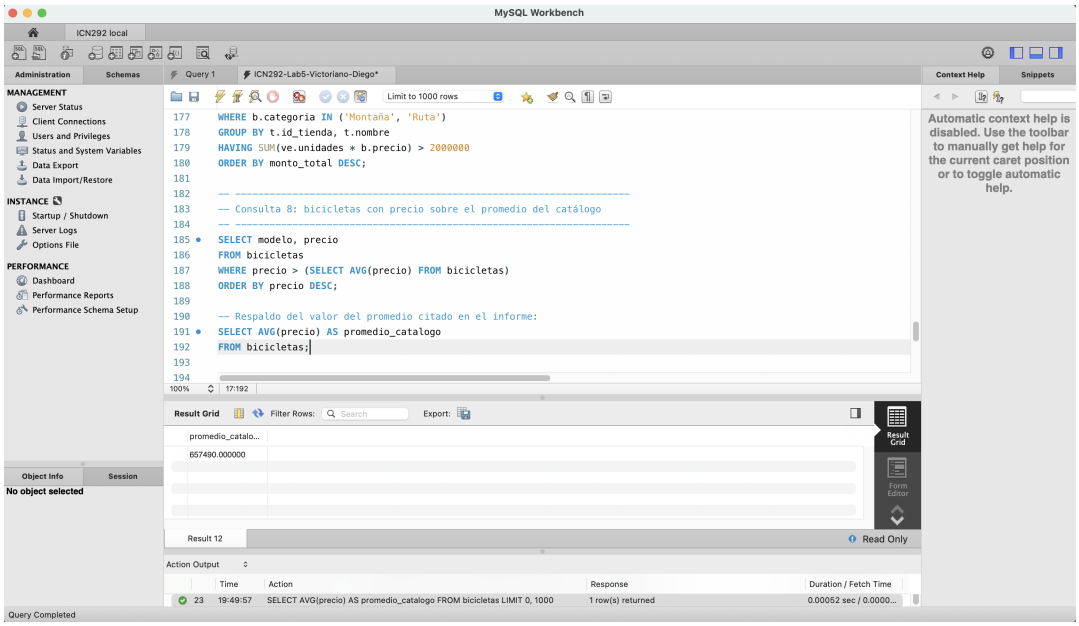


Figura 9: Verificación del promedio del catálogo en MySQL Workbench.

Consulta 9: vendedores sobre el promedio de unidades de la cadena

```sql
SELECT v.nombre,
       AVG(ve.unidades) AS promedio_unidades
FROM vendedores v
JOIN ventas ve ON v.id_vendedor = ve.id_vendedor
GROUP BY v.id_vendedor, v.nombre
HAVING AVG(ve.unidades) > (SELECT AVG(unidades) FROM ventas)
ORDER BY promedio_unidades DESC, v.nombre;
```

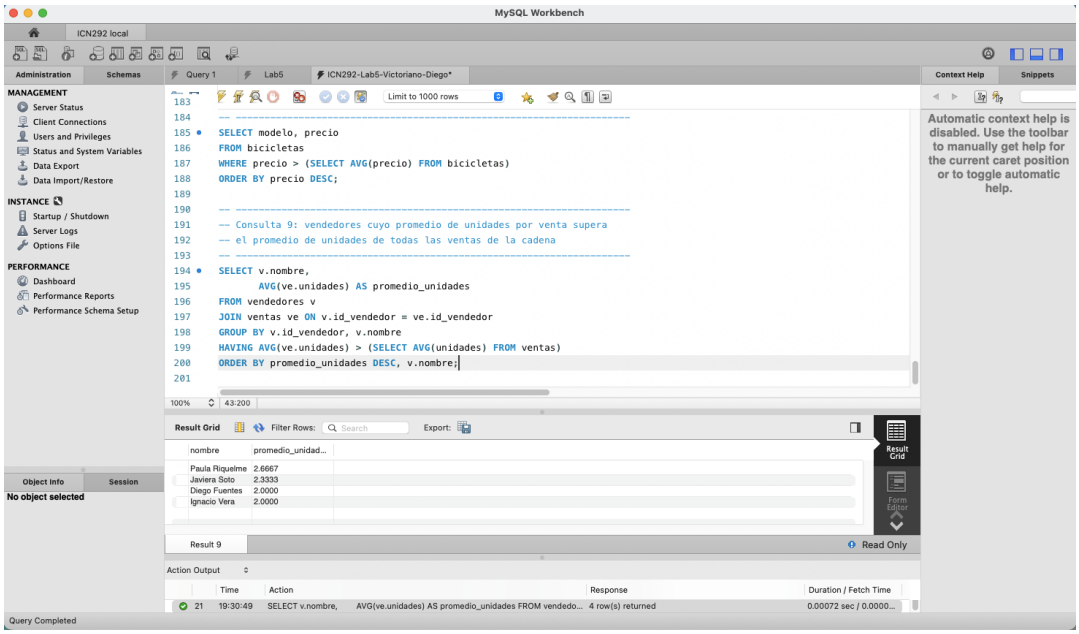


Figura 10: Resultado de la consulta 9 en MySQL Workbench.

Consulta 10: monto total vendido por vendedor

```sql
SELECT v.nombre,
       SUM(ve.unidades * b.precio) AS monto_total
FROM vendedores v
LEFT JOIN ventas     ve ON v.id_vendedor    = ve.id_vendedor
LEFT JOIN bicicletas b  ON ve.id_bicicleta  = b.id_bicicleta
GROUP BY v.id_vendedor, v.nombre
ORDER BY monto_total DESC, v.nombre;
```

Fernanda Lillo y Rubén Castro muestran NULL y no cero porque, al no tener ventas, el `LEFT JOIN` les deja `unidades` y `precio` en NULL, y `SUM` de puros NULL devuelve NULL. Para que mostraran cero habría que envolver la suma en `IFNULL(SUM(ve.unidades * b.precio), 0)`.

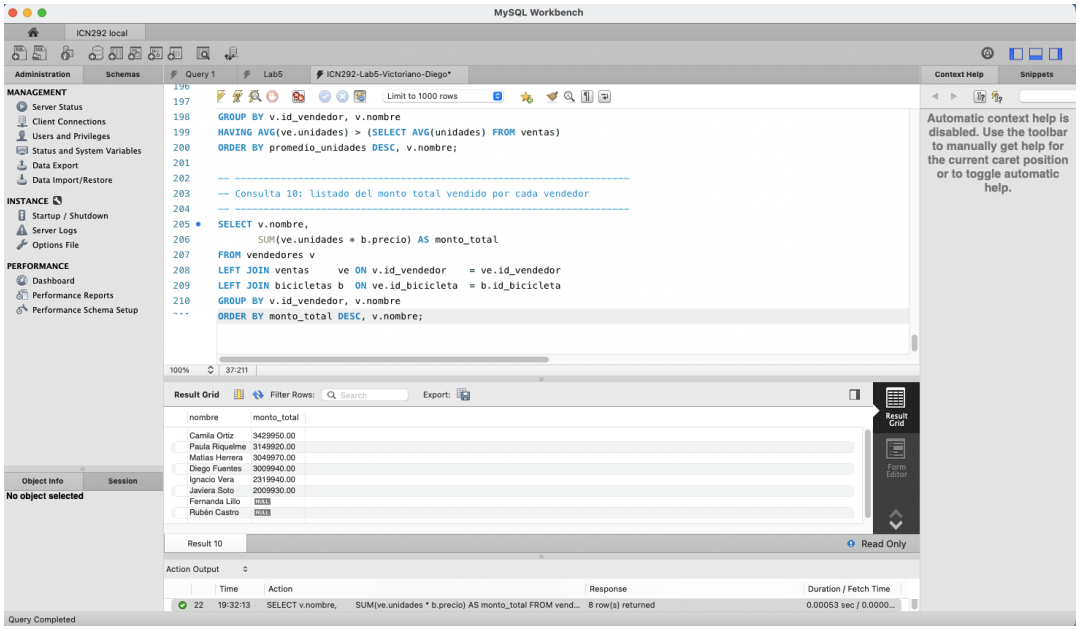


Figura 11: Resultado de la consulta 10 en MySQL Workbench.